NLP Revision Notes: Text Preprocessing to Word2Vec

One revision sheet from tokenization to Average Word2Vec: definitions, worked examples, code, comparisons and interview answers.

Oct 24, 2026 · @vishal meena

Tokenization | Stemming | Lemmatization | Stopwords | One-hot | Bag of Words | TF-IDF | Word2Vec

1 One-page cheat sheet (1 of 3)

Topic	What it does	Remember
Tokenization	Splits text into sentences or words	`sent_tokenize`, `word_tokenize`
Stemming	Cuts affixes by rules down to a stem	Fast, can give non-words (histori)
Lemmatization	Dictionary lookup gives a valid root word	Slower, needs POS (pos='v')
Stopwords	Removes very common words	Keep negations (not) for sentiment
POS tagging	Tags each word noun, verb, adjective...	`pos_tag(list_of_words)`
NER	Finds PERSON, ORGANIZATION, GPE, date, money	`pos_tag`, then `ne_chunk`

1 One-page cheat sheet (2 of 3)

Topic	What it does	Remember
One-hot	One vector per word with a single 1	Sparse, variable size, no meaning
Bag of Words	One count vector per sentence	Fixed size, loses word order
TF-IDF	BoW weighted by word importance	TF x IDF, a word in every sentence gets 0
Word2Vec	Neural network learns dense word vectors	CBOW or Skip-gram, 300 dims
Average Word2Vec	Mean of the word vectors in a sentence	One fixed-size vector per document

1 One-page cheat sheet (3 of 3)

Standard flow

Clean the text (lowercase, tokenize, remove stopwords, stem or lemmatize), vectorize it (BoW, TF-IDF or Word2Vec), then train a model.

Evolution of vectorization

One-hot, then Bag of Words, then TF-IDF (all count-based), then Word2Vec (learned, dense, captures meaning). Each step fixes problems of the one before.

2 Why NLP, the roadmap and use cases (1 of 2)

ML models train on math, so inputs must be numbers. Text (email subject, email body) cannot go in directly. NLP cleans text and converts it into vectors that keep its meaning. Example: spam vs ham classification.

Roadmap, bottom to top

Python, text preprocessing part 1 (tokenization, stemming, lemmatization, stopwords), part 2 (Bag of Words, TF-IDF, unigrams, bigrams), part 3 (Word2Vec, Average Word2Vec), then RNN, LSTM, GRU, then Transformers and BERT. Accuracy and model size both grow going up.

2 Why NLP, the roadmap and use cases (2 of 2)

Libraries

NLTK and spaCy for classic NLP, Gensim for Word2Vec, TensorFlow or PyTorch for deep learning. NLTK is a learning toolkit with many algorithms to choose from. spaCy is a faster, production-oriented pipeline with pretrained models.

Use cases

Gmail spell correction, auto-complete and smart replies, Google Translate, search, Hugging Face models (question answering, summarization, classification, translation), voice assistants (Alexa, Google Assistant), spam detection, sentiment analysis, chatbots, recommendations.

3 Basic terminology

Term	Meaning
Corpus	The whole paragraph or dataset of text
Document	One sentence (or one text sample) in the corpus
Vocabulary	The set of unique words in the corpus
Word	An individual word occurrence
Token	A unit produced by tokenization (sentence or word)

3 Terminology example and tokenization

Example

I like to drink apple juice. My friend likes mango juice. That is 2 sentences, 11 words and 10 unique words (juice repeats, like and likes count as different). Vocabulary size = 10.

Tokenization

Converting a paragraph or sentence into tokens (paragraph to sentences, or sentence to words). It is the first preprocessing step because every word must later become a vector.

4.1 Tokenization in NLTK: code

```
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize, wordpunct_tokenize
from nltk.tokenize import TreebankWordTokenizer
corpus = 'Hello welcome to the NLP tutorials. Please do watch the entire course!
To become expert in NLP.'
sent_tokenize(corpus)  # list of sentences
word_tokenize(corpus)  # words, punctuation as separate tokens
wordpunct_tokenize(corpus)  # splits ALL punctuation, even apostrophes (Krish's
becomes Krish, ', s)
TreebankWordTokenizer().tokenize(corpus)
```

4.1 Tokenization in NLTK: the four tokenizers

`sent_tokenize` uses the Punkt tokenizer and splits on full stops, exclamation marks and similar sentence ends.

`word_tokenize`: every word is a token, and punctuation (comma, full stop) becomes its own token.

`wordpunct_tokenize`: more aggressive, splits every punctuation mark including the apostrophe.

`TreebankWordTokenizer`: a full stop in the middle of the text stays attached to the previous word, only the final full stop is separated.

In practice `sent_tokenize` and `word_tokenize` cover most cases.

4.2 Stemming

Reducing a word to its word stem by stripping prefixes or suffixes. Eating, eats, eaten all map to eat, so the number of features drops. Typical task: classify a review as positive or negative, or mail as spam or ham.

- **PorterStemmer:** the basic stemmer.
- **RegexpStemmer:** you give a regex and a minimum length, and it removes whatever matches. Example regex: `ing$|s$|e$|able$` (the dollar sign means only at the end of the word).
- **SnowballStemmer:** better than Porter, supports many languages (`SnowballStemmer('english')`), also lowercases the output.

4.2 Stemming: code

```
from nltk.stem import PorterStemmer, RegexpStemmer, SnowballStemmer
porter = PorterStemmer()
snow = SnowballStemmer('english')
reg = RegexpStemmer('ing$|s$|e$|able$', min=4)
for w in
['eating','eats','eaten','writing','programming','history','finally','final
ized']:
    print(w, porter.stem(w), snow.stem(w))
```

Disadvantage: the stem may not be a valid word and the meaning can change. Even Snowball fails on some words (history, goes). Advantage: fast.

4.2 Stemming: Porter outputs

Word	Porter output	Comment
eating, eats	eat	good
eaten	eaten	unchanged
writing	write	good
programming, programs	program	good
finally, finalized	final	good
history	histori	not a real word
congratulations	congratul	meaning lost
fairly	fairli	Snowball gives fair
sportingly	sportingli	Snowball gives sport

4.3 Lemmatization

Like stemming, but the output (the lemma) is a valid root word with the same meaning, found by looking the word up in the WordNet corpus. Eating, eats, eaten give eat. Goes gives go.

```
from nltk.stem import WordNetLemmatizer
lem = WordNetLemmatizer()  # needs nltk.download('wordnet')
lem.lemmatize('going')  # going (default pos='n', treated as a noun)
lem.lemmatize('going', pos='v')  # go
lem.lemmatize('goes', pos='v')  # go
```

4.3 Lemmatization: POS argument and use

POS argument: n = noun (default), v = verb, a = adjective, r = adverb.

With the default noun tag most verbs are not reduced (eating stays eating). With pos='v': eating gives eat, writing gives write, history stays history (valid, unlike Porter).

Slower than stemming because every word is looked up in WordNet. Noticeable on large text.

Use for chatbots, Q&A, text summarization: anywhere the exact meaningful word matters.

4.4 Stemming vs lemmatization

	Stemming	Lemmatization
Method	Rule-based chopping	WordNet dictionary lookup
Output	Stem, may be invalid	Lemma, always a valid word
Speed	Fast	Slower
Needs POS	No	Yes, for good results
Use for	Spam, review classification	Chatbots, Q&A, summarization

4.5 Stopwords

Stopwords are very common words (I, the, he, she, of, to) that carry little meaning for tasks like spam or sentiment detection. Removing them shortens the text.

NLTK has stopword lists for many languages (English, German, French, Arabic and others).

Some stopwords matter: not, couldn't, aren't can flip the sentiment. Build a custom stopword list when needed.

4.5 The full cleaning pipeline: code

```
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import sent_tokenize, word_tokenize
# nltk.download('stopwords'); nltk.download('punkt'); nltk.download('wordnet')
lem = WordNetLemmatizer()
stop = set(stopwords.words('english'))
sentences = sent_tokenize(paragraph)
for i in range(len(sentences)):
    words = word_tokenize(sentences[i])
    words = [lem.lemmatize(w.lower(), pos='v') for w in words if w.lower() not in stop]
    sentences[i] = ' '.join(words)
```

Steps: paragraph to sentences, sentence to words, drop stopwords, stem or lemmatize what is left, join back with spaces. Use a set for the stopword list (fast lookup). Lowercase so Boy and boy are one word (Snowball lowercases by itself, the WordNet lemmatizer does not).

4.6 POS tagging

Labels each word with its part of speech. It gives the right lemma (going as verb vs noun) and feeds NER.

```
# nltk.download('averaged_perceptron_tagger')
words = 'Taj Mahal is a beautiful Monument'.split()
nltk.pos_tag(words)
# [('Taj','NNP'),('Mahal','NNP'),('is','VBZ'),('a','DT'),('beautiful','JJ'),
('Monument','NN')]
```

`pos_tag` needs a list of words. Passing a raw string tags every character.

4.6 POS tagging: common tags

Common tags: NN noun, NNS plural noun, NNP proper noun (names, places), NNPS plural proper noun, VB verb, VBD past tense verb, VBZ 3rd person singular verb, JJ adjective, RB adverb (RBR comparative like better, RBS superlative), PRP personal pronoun (I, he, she), DT determiner, CC coordinating conjunction, CD cardinal digit, IN preposition, EX existential there, FW foreign word.

4.7 Named Entity Recognition (NER)

Finds named entities: PERSON, ORGANIZATION, GPE (geo-political location), date and time, money. Example: The Eiffel Tower was built from 1887 to 1889 by French engineer Gustave Eiffel. Gustave Eiffel is tagged as a person, and the tower and French are picked up as entities.

```
# nltk.download('maxent_ne_chunker'); nltk.download('words')
words = nltk.word_tokenize(sentence)
tagged = nltk.pos_tag(words)
tree = nltk.ne_chunk(tagged)
tree.draw()  # draws the entity tree
```

Flow: tokenize, POS tag, `ne_chunk` .

5 Turning text into vectors: the big picture

A model needs one numeric vector per word or per document. Two families:

Family	Techniques	Idea
Count or frequency based	One-hot, Bag of Words, TF-IDF	Vector from word presence, counts or weights. Sparse
Learned (deep learning trained)	Word2Vec (CBOW, Skip-gram)	A neural network learns dense vectors where similar words sit close together

Five questions to ask about any technique: Is the input size fixed? Is it sparse? Does it capture meaning? Does it keep word order? How does it handle out-of-vocabulary (OOV) words?

6 One-hot encoding

Example corpus: D1 = the food is good, D2 = the food is bad, D3 = pizza is amazing. Vocabulary (7): the, food, is, good, bad, pizza, amazing.

Every word becomes a 7-dimensional vector with a single 1 at its own position: the = [1,0,0,0,0,0,0], food = [0,1,0,0,0,0,0], is = [0,0,1,0,0,0,0], and so on. D1 (4 words) is a 4 x 7 matrix, D2 is 4 x 7, D3 (3 words) is 3 x 7.

Advantage: easy to implement (sklearn `OneHotEncoder`, pandas `pd.get_dummies`).

6 One-hot: disadvantages (classic interview question)

1. Sparse matrix: mostly zeros, which leads to overfitting (great accuracy on training data, poor on new data).
2. No fixed-size input: D1 is 4 x 7 but D3 is 3 x 7, while ML algorithms need a fixed number of features.
3. No semantic meaning: food, pizza and burger are [1,0,0], [0,1,0], [0,0,1], all at equal distance, so similarity and importance are not captured.
4. Out-of-vocabulary: a new test word (burger in Burger is bad) is not in the vocabulary and cannot be encoded.
5. With a vocabulary of 50k words every vector has 50k dimensions, almost all zeros.

Because of this one-hot is not used on its own for NLP problems. Bag of Words and TF-IDF fix some of these points.

7 Bag of Words (BoW): steps

Example data (all positive, label 1): he is a good boy, she is a good girl, boy and girl are good.

1. Lowercase everything (so Boy and boy are one word).
2. Remove stopwords. Sentences become S1 = good boy, S2 = good girl, S3 = boy girl good.
3. Build the vocabulary with the frequency of each word, sorted high to low: good = 3, boy = 2, girl = 2.
4. Choose the features. You may keep only the top 10 or top 20 most frequent words and drop words that appear once.
5. Convert each sentence to a vector over the chosen features (good, boy, girl): S1 = [1, 1, 0], S2 = [1, 0, 1], S3 = [1, 1, 1].
6. Train the ML model on these vectors with the output label.

7 BoW: binary BoW and advantages

Binary BoW vs normal BoW: if good appears twice in a sentence, normal BoW stores 2 (the count), binary BoW forces it to 1 (present or not).

Advantages

- Simple and intuitive.
- Fixed-size input (the vocabulary size) for every sentence, so ML models can be trained. This fixes one-hot problem 2.

7 BoW: disadvantages

1. Still sparse (a 50k vocabulary gives 50k-dimensional vectors), so overfitting risk remains.
2. Word order is lost. The same words in another order give the same vector, so meaning can change.
3. Out-of-vocabulary words in test data (school) are ignored.
4. Meaning is not captured. Every present word gets the same value (1), so the most important word is not identified.
5. Opposite sentences look close: the food is good and the food is not good differ in one position only, so cosine similarity calls them near each other, though the meanings are opposite.

7 N-grams

Unigram = single words, bigram = two consecutive words, trigram = three. With bigrams, not good becomes one feature, which restores a little word order and partly fixes the negation problem. Cost: vocabulary and sparsity grow fast. In sklearn: `CountVectorizer(ngram_range=(1,2))`.

8 TF-IDF: the formulas

TF-IDF = Term Frequency x Inverse Document Frequency

TF = (times the word appears in the sentence) / (number of words in the sentence)

IDF = ln(number of sentences / number of sentences containing the word)

TF-IDF = TF x IDF

8 TF-IDF: worked example

S1 = good boy, S2 = good girl, S3 = boy girl good

	good	boy	girl
TF in S1	1/2	1/2	0
TF in S2	1/2	0	1/2
TF in S3	1/3	1/3	1/3
IDF	ln(3/3) = 0	ln(3/2) = 0.405	ln(3/2) = 0.405

TF-IDF vectors over (good, boy, girl)

S1 = [0, 1/2 x 0.405, 0] = [0, 0.203, 0]

S2 = [0, 0, 0.203]

S3 = [0, 1/3 x 0.405, 1/3 x 0.405] = [0, 0.135, 0.135]

8 TF-IDF: advantages and disadvantages

Advantages

1. Intuitive and easy to implement.
2. Fixed-size input (vocabulary size), same as BoW.
3. Word importance is captured, the main gain over BoW. A word present in every sentence (good) gets 0 because it does not help tell sentences apart. A word concentrated in few sentences gets a higher score. This gives the model better context and usually better accuracy.

Disadvantages

1. Still sparse (many zeros).
2. Out-of-vocabulary words in test data are ignored, because features come from the training vocabulary.
3. Word order and deeper meaning are still not captured.

8 TF-IDF: a note on sklearn

Note: sklearn's `TfidfVectorizer` uses a smoothed IDF, ln((1+N)/(1+df)) + 1, and L2-normalizes each row, so its numbers differ from the hand calculation above.

9 Word embeddings: the idea

A word embedding is a dense vector of real numbers for a word, learned so that words with similar meaning have vectors close together in the vector space.

Example: happy and excited get nearby vectors, while angry (opposite of happy) lands far away. To plot high-dimensional vectors in 2D, reduce them with PCA (an unsupervised dimensionality reduction technique).

Count-based methods (one-hot, BoW, TF-IDF) give sparse vectors with many drawbacks. Word2Vec, a model trained with deep learning, fixes most of them: dense vectors, meaning captured, no huge sparse matrices.

9 Feature representation (intuition)

Every vocabulary word is described by numbers over a set of features, each number showing how strongly the word relates to that feature. Illustrative values (a real model learns 100 to 300 features that have no human names):

Word	Gender	Royal	Age	Food
boy	-1.00	0.01	0.03	0.00
girl	1.00	0.02	0.02	0.00
king	-0.92	0.95	0.75	0.00
queen	0.93	0.96	0.68	0.00
apple	0.00	-0.02	0.50	0.91
mango	0.00	0.02	0.50	0.92

9 Reading the feature table

- boy and girl are opposite on gender but unrelated to royal (values near 0).
- king and queen are strongly royal and opposite on gender.
- apple and mango have high food values, so their vectors are close.
- Similar rows mean similar vectors, so similar words sit close together.

Famous analogy: remove the male direction from king, add the female direction, and the nearest word is queen.

king − man + woman ≈ queen

9 Cosine similarity and cosine distance

$$\cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|)$$

where $A \cdot B$ is the dot product and $\|A\|$, $\|B\|$ are the lengths of the vectors

$$\text{distance} = 1 - \cos(\theta)$$

Cosine compares direction (angle), not vector length.

9 Cosine similarity: angle table

Angle between vectors	cos	Distance	Meaning
0 degrees	1	0	Same direction, same word
45 degrees	0.707	0.293	Fairly similar
90 degrees	0	1	Unrelated
180 degrees	-1	2	Opposite

Used to find similar words, to compare sentence vectors, and in recommendations: an Avengers movie vector sits near an Iron Man vector because both are action and comic.

10 Word2Vec

Word2Vec (Google, Tomas Mikolov et al., 2013) trains a shallow neural network on a large corpus to learn word associations. Every distinct word gets a dense vector (for example 300 numbers). Trained vectors detect synonyms, opposites and analogies, and can suggest words for a partial sentence. Training uses ordinary ANN ideas: loss function, backpropagation, optimizer.

Two architectures: **CBOW** (Continuous Bag of Words) and **Skip-gram**.

10.1 Creating training data with a window

Corpus: iNeuron company is related to data science.
Vocabulary size V = 7 (iNeuron, company, is, related, to, data, science).

Choose a window size, here 5. Use an odd number so the centre word has the same number of words before and after it.

Take 5 words, the middle one is the centre word and the other 4 are the context. Slide the window one word at a time.

10.1 Window examples

Window	Context words	Centre word
1	iNeuron, company, related, to	is
2	company, is, to, data	related
3	is, related, data, science	to

Every word enters the network as a one-hot vector of size V = 7.
No labels are needed: the text itself supplies inputs and outputs.

10.2 CBOW: context words in, centre word out (1 of 2)

- **Input layer:** the context words, each a one-hot vector of V = 7 values.
- **Hidden layer:** N neurons, where N is the embedding dimension (5 in this toy example, 300 in Google's model). The context word vectors are averaged here.
- **Output layer:** V neurons with softmax, giving the probability of each vocabulary word being the centre word.
- **Weights:** input to hidden is V x N (7 x 5), hidden to output is N x V (5 x 7), both initialised randomly.

10.2 CBOW: training and the word vector (2 of 2)

Training: forward pass, compare predicted y-hat with the true one-hot y, compute the loss, backpropagate, repeat until the loss is minimal.

After training, the weights from a word's input node to the hidden layer are that word's vector: row i of the V x N matrix is the N-number vector of word i. For example iNeuron becomes a 5-number vector like [0.92, 0.94, 0.25, 0.36, 0.45] (illustrative).

10.3 Skip-gram: centre word in, context words out

Same network and training, with the direction reversed. The input is the centre word (one-hot, V values), the hidden layer has N neurons, and the output layer predicts each context word with softmax over V. Example: input is, predict iNeuron, company, related, to. Weights: V x N and N x V as before. Forward pass (weights, bias, activation), softmax output, loss between y and y-hat, backpropagation until the loss is minimal.

10.4 CBOW vs Skip-gram

	CBOW	Skip-gram
Input	Context words	Centre word
Output	Centre word	Context words
Training speed	Faster	Slower
Strength	Frequent words, smoother vectors	Rare words, better quality on large corpora
Typical choice	Quick training	Large datasets

10.5 Improving Word2Vec quality and practical facts

Improving quality

- More training data.
- Larger vector dimension (hidden layer size), typically 100 to 300.
- Tune window size, `min_count` and epochs.

Practical facts

Softmax over a large vocabulary is expensive, so real implementations use negative sampling or hierarchical softmax.

Pre-trained model `word2vec-google-news-300`: trained on Google News (about 100 billion words), 3 million words and phrases, 300 dimensions per word, about 1.6 GB. You can train your own with Gensim, but it needs a lot of data.

11 Average Word2Vec

Problem: Word2Vec gives one 300-number vector per word, so a sentence of n words becomes n x 300 numbers. A classifier needs one fixed-size vector per sentence.

Solution: average the word vectors of the sentence. The result is one 300-number vector per document that still carries semantic information.

$$v(\text{sentence}) = (1/n) \times [\, v(\text{word 1}) + v(\text{word 2}) + \ldots + v(\text{word n}) \,]$$

that is, add up the n word vectors and divide by n

11 Average Word2Vec: example and drawbacks

Example: the food is good becomes the mean of the vectors for the, food, is, good: one 300-number vector, paired with its label (1 or 0) to train the model.

Drawbacks: word order is lost, important words are diluted by common ones, and words missing from the model's vocabulary are skipped.

12 Word2Vec with Gensim: pre-trained model

```
import gensim.downloader as api
wv = api.load('word2vec-google-news-300')  # about 1.6 GB, download once (Colab is convenient)
wv['king'].shape  # (300,)
wv.most_similar('cricket')  # cricketing 0.83, cricketers 0.81, test_cricket 0.80 ...
wv.most_similar('happy')  # glad, pleased, ecstatic, overjoyed, thrilled, satisfied, proud, delighted
wv.similarity('hockey', 'sports')  # about 0.53
vec = wv['king'] - wv['man'] + wv['woman']
wv.most_similar([vec])  # king first (it is in the input), then queen, monarch, princess ...
wv.most_similar(positive=['king', 'woman'], negative=['man'])  # queen on top
```

12 Word2Vec with Gensim: train from scratch

Train from scratch:

```
from gensim.models import Word2Vec
sentences = [['i','love','nlp'], ['nlp','is','fun']]  # list of token lists
model = Word2Vec(sentences, vector_size=100, window=5, min_count=1, sg=0,
epochs=10)
# sg=0 CBOW, sg=1 Skip-gram (Gensim 3 used size instead of vector_size)
model.wv['nlp']  # 100-number vector
model.wv.most_similar('nlp')
```

13 Master comparison table

Property	One-hot	BoW	TF-IDF	Word2Vec	Average Word2Vec
Unit encoded	Each word	Sentence	Sentence	Each word	Sentence
Fixed-size output	No	Yes (vocab size)	Yes (vocab size)	Yes per word (e.g. 300)	Yes (e.g. 300)
Sparse or dense	Sparse	Sparse	Sparse	Dense	Dense
Word order	Not kept	Lost	Lost	Context learned in training	Lost
Meaning captured	No	No	No	Yes	Yes (averaged)
Word importance	No	No	Yes	No	No
Out-of-vocabulary	Fails	Ignored	Ignored	No vector	Skipped
Needs training	No	No	No	Yes (or pre-trained)	Uses Word2Vec

14 Interview Q&A: Preprocessing (1 of 3)

Why can't ML models take raw text?

Models train on math, so they need numbers. NLP turns text into vectors that keep its meaning.

Stemming vs lemmatization?

Stemming chops affixes by rules (fast, may give invalid words like histori). Lemmatization looks words up in WordNet and returns a valid root (slower, needs POS). Use stemming for spam or sentiment, lemmatization for chatbots, Q&A and summarization.

Why does lemmatization need the POS tag?

The default is noun, so verbs stay unchanged (going stays going). With pos='v' you get go.

14 Interview Q&A: Preprocessing (2 of 3)

Porter vs Snowball?

Snowball is the improved version: multi-language, lowercases output, better stems (fair and sport where Porter gives fairli and sportingli).

When should you not remove stopwords?

For sentiment and other negation-sensitive tasks (not, never, couldn't), which flip meaning.

word_tokenize vs wordpunct_tokenize vs Treebank?

`word_tokenize` splits words and punctuation. `wordpunct_tokenize` splits every punctuation mark, apostrophes included. Treebank keeps a mid-text full stop attached to the previous word and separates only the final one.

14 Interview Q&A: Preprocessing (3 of 3)

POS tagging vs NER?

POS gives each word's grammatical role (NN, VB, JJ). NER finds real-world entities (PERSON, ORGANIZATION, GPE, date, money).

Why lowercase text?

So Boy and boy are one feature, which shrinks the vocabulary.

Usual cleaning order?

Lowercase, remove punctuation or numbers if needed, tokenize, remove stopwords, stem or lemmatize, then vectorize.

14 Interview Q&A: One-hot, BoW, TF-IDF (1 of 3)

Drawbacks of one-hot?

Sparse (overfitting), variable input size, no semantic similarity (all words equidistant), out-of-vocabulary words, huge dimensions.

How does BoW work and what is binary BoW?

Build the vocabulary, then represent each sentence by word counts over it. Binary BoW stores only 0 or 1 (present or not).

Why does BoW fail on the food is good vs the food is not good?

Only one position differs, so the vectors are close (high cosine similarity) though the meanings are opposite. Word order and context are lost.

14 Interview Q&A: One-hot, BoW, TF-IDF (2 of 3)

What are n-grams and their trade-off?

Sequences of n consecutive words. Bigrams keep a little order (not good is one feature) but vocabulary and sparsity explode.

TF-IDF formula and intuition?

TF = count in sentence / words in sentence. IDF = ln(total sentences / sentences containing the word). A word common to all sentences gets IDF 0 (no information), a word concentrated in a few sentences scores high.

TF-IDF vs BoW?

BoW treats every present word equally. TF-IDF weights words by importance, so context improves.

14 Interview Q&A: One-hot, BoW, TF-IDF (3 of 3)

What is the out-of-vocabulary problem and how is it handled?

A word seen at test time but absent from the training vocabulary. One-hot, BoW and TF-IDF ignore it or fail. Options: an UNK token, a larger vocabulary, subword models (FastText, BPE).

Why does sparsity cause overfitting and how do you reduce it?

Very high-dimensional, mostly-zero features let the model memorise the training data. Reduce with `max_features`, `min_df`, stopword removal, SVD, or dense embeddings.

Why cosine similarity instead of Euclidean distance for text?

Cosine compares direction, so document length does not distort similarity.

14 Interview Q&A: Word2Vec (1 of 7)

What is a word embedding?

A dense, learned vector for a word where similar meanings are close in vector space.

Count-based vs learned vectorization?

Count-based (one-hot, BoW, TF-IDF) come from presence, counts or weights and are sparse. Learned (Word2Vec) come from a neural network and are dense and meaningful.

What is Word2Vec?

A shallow neural network from Google (Mikolov, 2013) that learns word vectors from a large corpus, in two flavours: CBOW and Skip-gram.

14 Interview Q&A: Word2Vec (2 of 7)

Explain CBOW.

Context words go in, the network predicts the centre word. The hidden layer averages the context vectors, and softmax gives a probability over the vocabulary.

Explain Skip-gram.

The centre word goes in, the network predicts the surrounding context words.

CBOW vs Skip-gram, which when?

CBOW is faster and good for frequent words. Skip-gram is slower but better for rare words and large corpora.

14 Interview Q&A: Word2Vec (3 of 7)

What decides the vector size, and what is window size?

Vector size = number of hidden layer neurons (for example 300). Window size = how many words around the centre are used as context. They are separate hyperparameters.

Why an odd window size?

The centre word then has the same number of words before and after it.

Weight matrix shapes?

Input to hidden is V x N, hidden to output is N x V (V = vocabulary size, N = embedding dimension).

Interview Q&A: Word2Vec (4 of 7)

How do you get a word's vector after training?

Take that word's row of the input-to-hidden weight matrix.

King - man + woman = ?

Approximately queen. Meaning is encoded as directions in the vector space.

Cosine similarity and distance?

$\cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|)$, distance $= 1 - \cos(\theta)$. 0 degrees gives distance 0 (same), 90 degrees gives 1 (unrelated).

14 Interview Q&A: Word2Vec (5 of 7)

How do you improve Word2Vec?

More data, larger embedding dimension, tuned window, `min_count` and epochs.

What is Average Word2Vec and why is it needed?

Classifiers need one fixed-size vector per document, so the word vectors of a sentence are averaged into one 300-number vector.

Drawbacks of Average Word2Vec?

Loses word order, dilutes important words, skips words missing from the model. Alternatives: TF-IDF-weighted average, Doc2Vec, transformer embeddings.

14 Interview Q&A: Word2Vec (6 of 7)

Pre-trained vs training from scratch?

Pre-trained (Google News, 300 dims, 3 million words and phrases) works out of the box. Train your own on domain text when your vocabulary is specialised and you have enough data.

Limitations of Word2Vec?

One static vector per word (bank in river bank and bank account is the same vector), no vector for unseen words, needs lots of data. Contextual models (ELMo, BERT) fix the first, FastText helps with the second. GloVe is another static embedding.

Interview Q&A: Word2Vec (7 of 7)

Why is Word2Vec training costly and how is it sped up?

The softmax runs over the whole vocabulary. Negative sampling and hierarchical softmax approximate it cheaply.

Word2Vec vs TF-IDF in one line?

TF-IDF is sparse and about word importance. Word2Vec is dense and about meaning and similarity.

15 Code reference: text cleaning function

```
import re
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
lem = WordNetLemmatizer()
stop = set(stopwords.words('english')) - {'not', 'no', 'nor'}  # keep
negations
def clean_text(text):
    text = re.sub(r'[^a-z ]', ' ', text.lower())
    tokens = word_tokenize(text)
    tokens = [lem.lemmatize(t, pos='v') for t in tokens if t not in stop
and len(t) > 1]
    return ' '.join(tokens)
```

15 Code reference: TF-IDF from scratch

```
import math
from collections import Counter
def tfidf(docs):
    docs = [d.split() for d in docs]
    N = len(docs)
    vocab = sorted({w for d in docs for w in d})
    df = {w: sum(w in d for d in docs) for w in vocab}
    idf = {w: math.log(N / df[w]) for w in vocab}
    rows = []
    for d in docs:
        c = Counter(d)
        rows.append([(c[w] / len(d)) * idf[w] for w in vocab])
    return vocab, rows
tfidf(['good boy', 'good girl', 'boy girl good'])
```

15 Code reference: cosine similarity, Average Word2Vec

```
import numpy as np
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
def avg_word2vec(tokens, wv):
    vecs = [wv[w] for w in tokens if w in wv.key_to_index]
    return np.mean(vecs, axis=0) if vecs else np.zeros(wv.vector_size)
```

15 Code reference: TF-IDF + Naive Bayes classifier

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, f1_score
X_train, X_test, y_train, y_test = train_test_split(texts, labels, test_size=0.2,
random_state=42)
tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1, 2))
Xtr = tfidf.fit_transform(X_train)  # fit on the training data only
Xte = tfidf.transform(X_test)  # only transform the test data
clf = MultinomialNB().fit(Xtr, y_train)
pred = clf.predict(Xte)
print(accuracy_score(y_test, pred), f1_score(y_test, pred))
```

16 Common errors and gotchas (1 of 2)

- `LookupError` for punkt, stopwords, wordnet, averaged_perceptron_tagger, maxent_ne_chunker or words: run `nltk.download` with the missing name. Recent NLTK versions (3.9 and later) name them punkt_tab, averaged_perceptron_tagger_eng and maxent_ne_chunker_tab.
- `pos_tag` on a string tags every character: pass a list of words.
- `RegexpStemmer` needs a regex as its first argument. Without the end-of-word anchor it removes matches anywhere in the word.
- Use `word.lower()` on each string, not on a list.

16 Common errors and gotchas (2 of 2)

- `WordNetLemmatizer` is case-sensitive: lowercase first, and pass the POS.
- Fit the vectorizer on the training data only and only transform the test data, otherwise information leaks from test to train.
- `wv[word]` raises `KeyError` for words not in the model: check `word in wv.key_to_index` first.
- Google News vectors are case-sensitive (Cricket and cricket differ).
- Gensim 4 uses `vector_size`, Gensim 3 used `size`.